

ROLL.NO:240701506

NAME: SHREYA KS

EXPERIMENT - 6

CPU SCHEDULING ALGORITHM

1. First-Come,First-Served CPU Scheduling Algorithm

AIM:

Write a C program to implement the First-Come, First-Served (FCFS) CPU Scheduling Algorithm. The program should take the number of processes, their Arrival Times (AT), and Burst Times (BT) as input. Calculate and display the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process, along with the Average Turnaround Time and Average Waiting Time.

To evaluate the performance of the algorithm, we use the following formulas:

- Completion Time (CT): The time at which a process finishes execution.
- Turnaround Time (TAT): Total time taken from arrival to completion.
- $TAT = CT - AT$
- Waiting Time (WT): The time a process spends waiting in the ready queue.
- $WT = TAT - BT$

Input Format

1. The first input line contains an integer representing the number of processes.
2. The next lines contain two integers for each process, where the first integer represents the Arrival Time (AT) and the second integer represents the Burst Time (BT).

Output Format

The output displays the Average Turnaround Time and the Average Waiting Time.

Example 1

Sample Input 1

```
3
0 2
1 2
2 3
```

Sample Output 1

```
Enter number of processes:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:
```

Average Turnaround Time: 3.33

Average Waiting Time: 1.00

Example 2

Sample Input 2

3
0 5
5 3
8 2

Sample Output 2

Enter number of processes:

Enter Arrival Time and Burst Time for P1:

Enter Arrival Time and Burst Time for P2:

Enter Arrival Time and Burst Time for P3:

Average Turnaround Time: 3.33

Average Waiting Time: 0.00

CODE:

```
Select Language : C (GCC 9.2.0)
1  #include <stdio.h>
2  int main(){
3      int n;
4      printf("Enter number of processes: \n");
5      scanf("%d", &n);
6      int AT[n], BT[n], CT[n], TAT[n], WT[n];
7      for(int i=0;i<n;i++){
8          printf("Enter Arrival Time and Burst Time for P%d: \n",i+1);
9          scanf("%d %d", &AT[i], &BT[i]);
10     }
11     int time=0;
12     for(int i=0;i<n;i++){
13         if(time<AT[i]){
14             time=AT[i];
15         }
16         time += BT[i];
17         CT[i]=time;
18         TAT[i]=CT[i]-AT[i];
19         WT[i]=TAT[i]-BT[i];
20     }
21     float avgTAT = 0, avgWT = 0;
22     for(int i=0;i<n;i++){
23         avgTAT += TAT[i];
24         avgWT += WT[i];
```

OUTPUT:

Custom Test Case

Custom Test

Success

Input :

3
0 2
1 2
2 3

Expected Output :

Enter number of processes:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:

Average Turnaround Time: 3.33
Average Waiting Time: 1.00

Output :

Enter number of processes:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:

Average Turnaround Time: 3.33
Average Waiting Time: 1.00

RESULT:

Thus the program to run First-Come,First-Served CPU Scheduling Algorithm as been complied successfully

2. Non-Preemptive Shortest Job First CPU Scheduling Algorithm

AIM:

Write a C program to implement the Non-Preemptive Shortest Job First (SJF) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Times (AT), and Burst Times (BT). At any given time, the process with the minimum burst time among the arrived processes must be executed first. Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process and find the overall average TAT and WT.

Core Logic

Selection: Out of all processes that have arrived by the current time, the one with the minimum Burst Time is chosen.

Formulae:

- $\text{Turnaround Time (TAT)} = \text{Completion Time (CT)} - \text{Arrival Time (AT)}$
- $\text{Waiting Time (WT)} = \text{Turnaround Time (TAT)} - \text{Burst Time (BT)}$

Input Format

1. The first line of input contains an integer representing the number of processes.

2. The next lines contain two integers for each process, where the first integer denotes the Arrival Time (AT) of the process and the second integer denotes the Burst Time (BT) of the process.

Output Format

The output displays the Average Turnaround Time and the Average Waiting Time.

Example 1

Sample Input 1

```
4
0 6
0 8
0 7
0 3
```

Sample Output 1

```
Enter number of processes:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:
Enter Arrival Time and Burst Time for P4:
Average Turnaround Time = 13.00
Average Waiting Time = 7.00
```

Example 2

Sample Input 2

```
3
0 3
5 2
5 1
```

Sample Output 2

```
Enter number of processes:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:
Average Turnaround Time = 2.33
Average Waiting Time = 0.33
```

CODE:

```
1  #include <stdio.h>
2  int main(){
3      int n;
4      printf("Enter number of processes: \n");
5      scanf("%d", &n);
6      int AT[n], BT[n], CT[n], TAT[n], WT[n];
7      int completed[n];
8      for(int i=0;i<n;i++){
9          completed[i]=0;
10     }
11     for(int i=0;i<n;i++){
12         printf("Enter Arrival Time and Burst Time for P%d: \n",i+1);
13         scanf("%d %d", &AT[i], &BT[i]);
14     }
15     int time = 0, done = 0;
16     while(done<n)
17     {
18         int idx=-1;
19         int minBT=100000;
20         for(int i=0;i<n;i++){
21             if(AT[i]<=time&&completed[i]==0)
22             {
23                 if(BT[i]<minBT)
24                     idx=i;
25             }
26         }
27         if(idx!=-1){
28             CT[idx]=0;
29             TAT[idx]=0;
30             WT[idx]=0;
31             time+=BT[idx];
32             completed[idx]=1;
33             done++;
34         }
35     }
36     double sumTAT=0, sumWT=0;
37     for(int i=0;i<n;i++){
38         sumTAT+=TAT[i];
39         sumWT+=WT[i];
40     }
41     printf("Average Turnaround Time = %.2f\n", sumTAT/n);
42     printf("Average Waiting Time = %.2f\n", sumWT/n);
43 }
```

output:

Input :

```
4
0 6
0 8
0 7
0 3
```

Expected Output :

```
Enter number of processes:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:
Enter Arrival Time and Burst Time for P4:

Average Turnaround Time = 13.00
Average Waiting Time = 7.00
```

Output :

```
Enter number of processes:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:
Enter Arrival Time and Burst Time for P4:

Average Turnaround Time = 13.00
Average Waiting Time = 7.00
```

RESULT: Thus the program to run Non-Preemptive Shortest Job First CPU Scheduling Algorithm as been compiled successfully.

3. Non-Preemptive Priority Scheduling algorithm

AIM:

Write a C program to implement the Non-Preemptive Priority Scheduling algorithm. The program should accept number of processes their Arrival Time (AT), Burst Time (BT), and Priority for each process. The process with the highest priority (lowest numerical value) must be executed first among those that have arrived. Calculate the Average Waiting Time and Average Turnaround Time.

Input Format

The first line contains an integer representing the number of processes.

The next lines contain three integers for each process, where

- The first integer denotes the Arrival Time (AT)
- The second integer denotes the Burst Time (BT)
- The third integer denotes the Priority of the process.

A lower numerical value indicates a higher priority.

Output Format

The output displays the Average Turnaround Time and the Average Waiting Time.

Example 1**Sample Input 1**

```
3
0 5 2
1 3 1
2 2 3
```

Sample Output 1

Enter number of processes:

Enter AT, BT and Priority for P1:

Enter AT, BT and Priority for P2:

Enter AT, BT and Priority for P3:

Average Turnaround Time = 6.67

Average Waiting Time = 3.33

Example 2**Sample Input 2**

```
4
0 4 3
0 3 1
2 2 4
```

5 1 2

Sample Output 2

Enter number of processes:

Enter AT, BT and Priority for P1:

Enter AT, BT and Priority for P2:

Enter AT, BT and Priority for P3:

Enter AT, BT and Priority for P4:

Average Turnaround Time = 5.25

Average Waiting Time = 2.75

CODE:

```
1  #include <stdio.h>
2
3  int main(){
4      int n;
5
6      printf("Enter number of processes: \n");
7      scanf("%d", &n);
8
9      int AT[n], BT[n], PR[n], CT[n], TAT[n], WT[n];
10     int completed[n];
11
12     for(int i=0;i<n;i++){
13         completed[i]=0;
14     }
15     for(int i=0;i<n;i++){
16         printf("Enter AT, BT and Priority for P%d: \n",i+1);
17         scanf("%d %d %d",&AT[i],&BT[i],&PR[i]);
18     }
19     int time = 0, done = 0;
20     while(done<n){
21         int idx = -1;
22         int highestPriority=100000;
23
24         for(int i=0;i<n;i++){
```

OUTPUT:

Input :

```
3
0 5 2
1 3 1
2 2 3
```

Expected Output :

```
Enter number of processes:
Enter AT, BT and Priority for P1:
Enter AT, BT and Priority for P2:
Enter AT, BT and Priority for P3:

Average Turnaround Time = 6.67
Average Waiting Time = 3.33
```

Output :

```
Enter number of processes:
Enter AT, BT and Priority for P1:
Enter AT, BT and Priority for P2:
Enter AT, BT and Priority for P3:

Average Turnaround Time = 6.67
Average Waiting Time = 3.33
```

RESULT:

Thus the program to run Non-Preemptive Priority Scheduling algorithm as been complied successfully.